# COMP5310: Principles of Data Science

## W10: Decision Tree

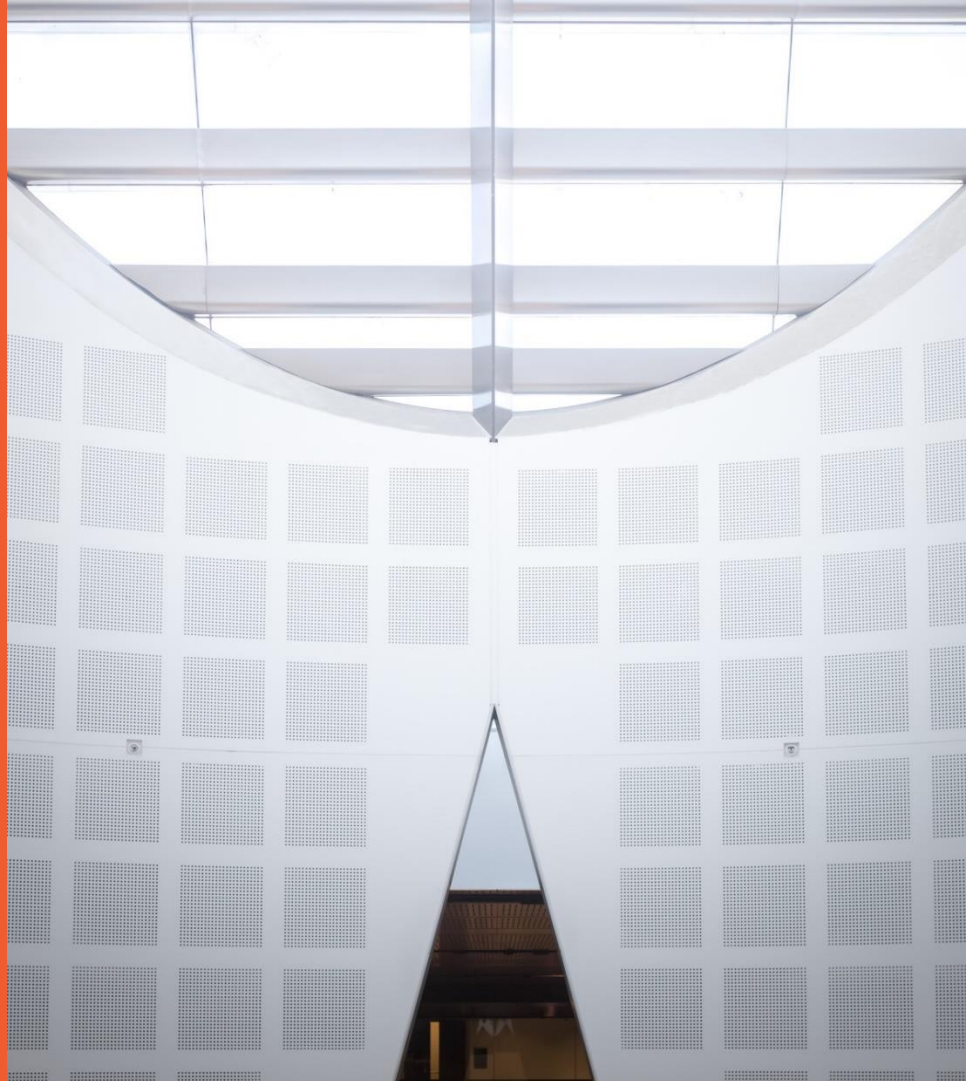
**Presented by**

A/Prof. Nguyen Tran

School of Computer Science

Based on slides by previous lecturers of this unit of study

THE UNIVERSITY OF SYDNEY



# Last week: Linear regression & logistic regression

**Objective**

Learn techniques for supervised machine learning, with tools in Python.

**Lecture**

- Simple linear regression
- Multiple linear regression
- Gradient descent
- Logistic regression

**Readings**

- Data Science from Scratch, Ch. 8, 14, 15, 16

**Exercises**

- sklearn: regression

# Supervised Learning:

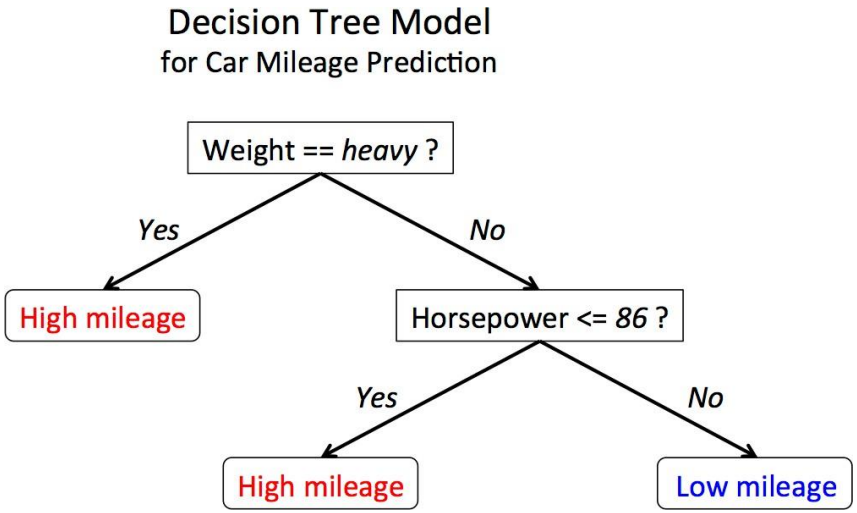
- We'll now focus on supervised machine learning techniques
  - ☑ *Simple linear regression*
  - ☑ *Multiple linear regression*
  - ☑ *Logistic regression*
  - **Decision tree**
  - Naïve Bayes

# DECISION TREE

THE UNIVERSITY OF SYDNEY

# Decision tree classification

- Maps observations to a target value by asking a series of questions
- Can be viewed as hierarchy of if/else statements.
  - Each **non-leaf** node is labelled with an attribute and corresponds to a test for the values of an attribute
  - Each leaf node is labelled with a class
- Resulting model is **intuitive** and **interpretable.**
- Ensembles of simple trees can do very well.




https://databricks.com/blog/2014/09/29/scalable-decision-trees-in-mllib.html

# A greedy ID3 algorithm for decision tree induction

- Start with a **single root node** containing all training examples
- For each node:
  - If all examples at this node belong to the same class:
    - mark this node as a leaf and label it with that class
  - Otherwise:
    - Compute heuristic or statistical measure (e.g. **Information Gain**) for each remaining candidate attributes
    - Select the best attribute (e.g. with the highest IG)
    - Label the current node with that attribute
    - Split the examples into child nodes:
      - **Categorical attributes**: one branch per value
      - **Continues attributes**: choose a split point a (e.g. $A < a, A \geq a$)
- Repeat the process for each child node
- Stop when:
  - All examples at a node belong to the same class; or
  - no attributes remain → label with **majority class**

# Example

- **Training data:** interviewee data.
- **Features:** Level , Lang, Tweets, PhD.
- **Class label:** Interviewed well.
- **New applicant:** A15 (Senior, R, No, No).
- We want to **predict whether A15 Interviewed well or not**

Training examples: 9 True/ 5 False

Class label

| Applicant | Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|---|
| A1 | Senior | Java | No | No | False |
| A2 | Senior | Java | No | Yes | False |
| A3 | Mid | Java | No | No | True |
| A4 | Junior | Python | No | No | True |
| A5 | Junior | R | Yes | No | True |
| A6 | Junior | R | Yes | Yes | False |
| A7 | Mid | R | Yes | Yes | True |
| A8 | Senior | Python | No | No | False |
| A9 | Senior | R | Yes | No | True |
| A10 | Junior | Python | Yes | No | True |
| A11 | Senior | Python | Yes | Yes | True |
| A12 | Mid | Python | No | Yes | True |
| A13 | Mid | Java | Yes | No | True |
| A14 | Junior | Python | No | Yes | False |
| **New data:** A15 | Senior | R | No | No | ? |

# Decision Tree

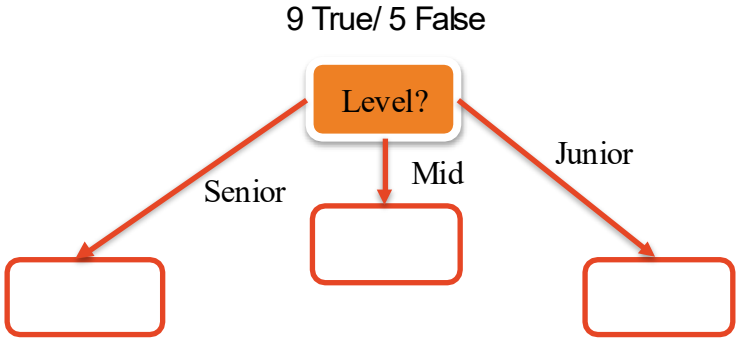
- Divide-and-conquer:
  - Choose attributes to split the data into subsets
  - Are they pure?(all True or all False)
    - If yes: stop
    - Otherwise: repeat
- Which attributes to choose?
- Let's try selecting "Level" attribute first.

Training examples: 9 True/ 5 False

Class label

| Applicant | Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|---|
| A1 | Senior | Java | No | No | False |
| A2 | Senior | Java | No | Yes | False |
| A3 | Mid | Java | No | No | True |
| A4 | Junior | Python | No | No | True |
| A5 | Junior | R | Yes | No | True |
| A6 | Junior | R | Yes | Yes | False |
| A7 | Mid | R | Yes | Yes | True |
| A8 | Senior | Python | No | No | False |
| A9 | Senior | R | Yes | No | True |
| A10 | Junior | Python | Yes | No | True |
| A11 | Senior | Python | Yes | Yes | True |
| A12 | Mid | Python | No | Yes | True |
| A13 | Mid | Java | Yes | No | True |
| A14 | Junior | Python | No | Yes | False |
| **New data:** A15 | Senior | R | No | No | ? |

# Decision Tree

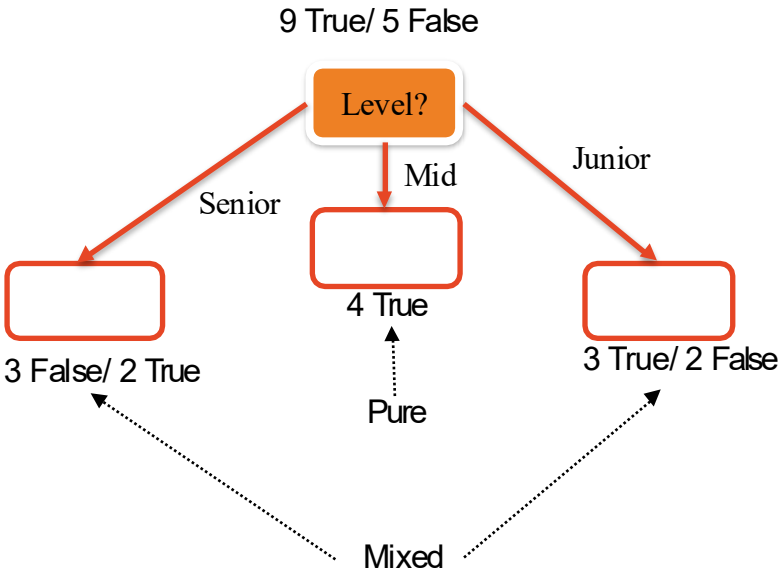



Training examples: 9 True/ 5 False

Class label

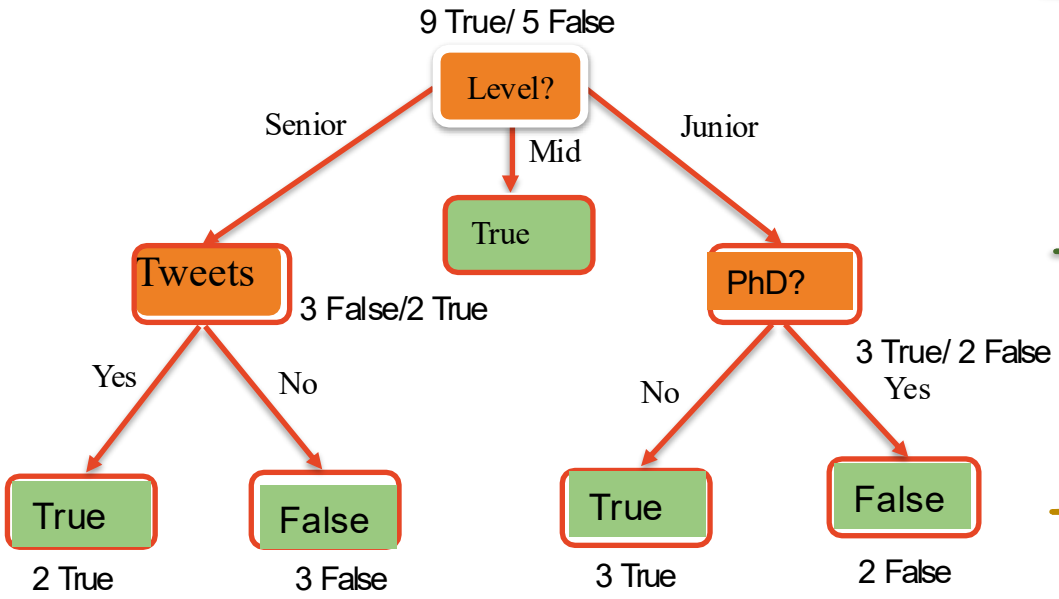
| Applicant | Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|---|
| A1 | Senior | Java | No | No | False |
| A2 | Senior | Java | No | Yes | False |
| A3 | Mid | Java | No | No | True |
| A4 | Junior | Python | No | No | True |
| A5 | Junior | R | Yes | No | True |
| A6 | Junior | R | Yes | Yes | False |
| A7 | Mid | R | Yes | Yes | True |
| A8 | Senior | Python | No | No | False |
| A9 | Senior | R | Yes | No | True |
| A10 | Junior | Python | Yes | No | True |
| A11 | Senior | Python | Yes | Yes | True |
| A12 | Mid | Python | No | Yes | True |
| A13 | Mid | Java | Yes | No | True |
| A14 | Junior | Python | No | Yes | False |
| **New data:** A15 | Senior | R | No | No | ? |

# Decision Tree



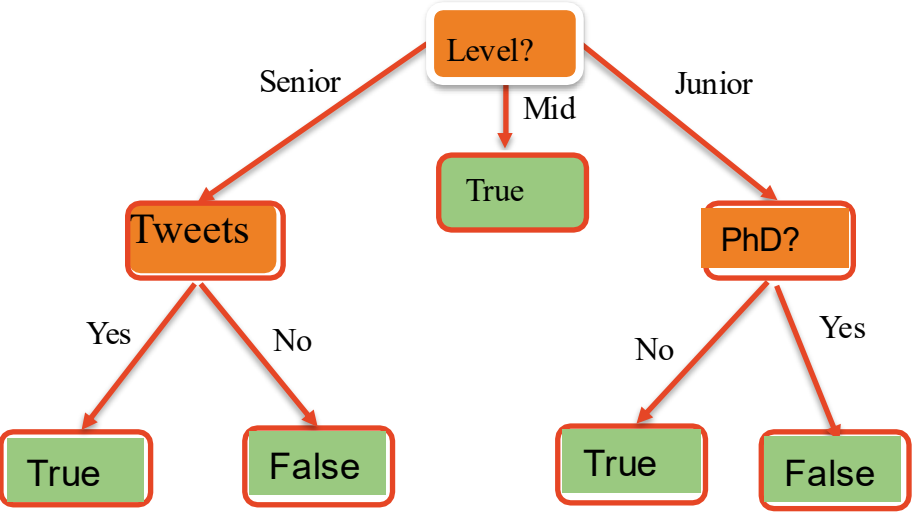

| Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|
| Senior | Java | No | No | False |
| Senior | Java | No | Yes | False |
| Senior | Python | No | No | False |
| Senior | R | Yes | No | True |
| Senior | Python | Yes | Yes | True |
| Mid | Java | No | No | True |
| Mid | R | Yes | Yes | True |
| Mid | Python | No | Yes | True |
| Mid | Java | Yes | No | True |
| Junior | Python | No | No | True |
| Junior | R | Yes | No | True |
| Junior | Python | Yes | No | True |
| Junior | R | Yes | Yes | False |
| Junior | Python | No | Yes | False |

# Decision Tree




| Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|
| Senior | Java | No | No | False |
| Senior | Java | No | Yes | False |
| Senior | Python | No | No | False |
| Senior | R | Yes | No | True |
| Senior | Python | Yes | Yes | True |
| Mid | Java | No | No | True |
| Mid | R | Yes | Yes | True |
| Mid | Python | No | Yes | True |
| Mid | Java | Yes | No | True |
| Junior | Python | No | No | True |
| Junior | R | Yes | No | True |
| Junior | Python | Yes | No | True |
| Junior | R | Yes | Yes | False |
| Junior | Python | No | Yes | False |

# Resulting Tree




| Applicant | Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|---|
| A1 | Senior | Java | No | No | False |
| A2 | Senior | Java | No | Yes | False |
| A3 | Mid | Java | No | No | True |
| A4 | Junior | Python | No | No | True |
| A5 | Junior | R | Yes | No | True |
| A6 | Junior | R | Yes | Yes | False |
| A7 | Mid | R | Yes | Yes | True |
| A8 | Senior | Python | No | No | False |
| A9 | Senior | R | Yes | No | True |
| A10 | Junior | Python | Yes | No | True |
| A11 | Senior | Python | Yes | Yes | True |
| A12 | Mid | Python | No | Yes | True |
| A13 | Mid | Java | Yes | No | True |
| A14 | Junior | Python | No | Yes | False |
| **A15** | **Senior** | **R** | **No** | **No** | False |

# INFORMATION GAIN

THE UNIVERSITY OF SYDNEY

# How does ID3 choose an attribute and split

- It chooses the attribute with the highest information gain.
- At each non-leaf node, the algorithm asks:
  - Which attribute gives the biggest reduction in uncertainty?
- Information Gain is calculated as the difference between the uncertainty before the split and the uncertainty after the split.
- Higher Information Gain means a better split
- Uncertainty is measured by entropy

# Information Gain (IG)

- Formally IG calculates the **change in entropy** after making a split based on the value of an attribute.

$$IG(Y|X) = H(Y) - H(Y|X)$$

- Where:
  - $Y$ is a class label.
  - $X$ is an attribute.
  - $H(Y)$ is the entropy of $Y$: uncertainty before splitting
  - $H(Y|X)$ is the conditional entropy of $Y$ given $X$: uncertainty after splitting

# Entropy

- To measure the **uncertainty** associated with data:

$$H(Y) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

- Where $p_i = p(Y = y_i)$, and $m$ is the number of classes.
- Interpretation:
  - Higher entropy => higher uncertainty.
  - Lower entropy => lower uncertainty.
- Entropy ranges from 0 (completely pure) to $log_2(k)$ ,where $k$ is the number of classes.
- For binary classification, the maximum entropy is 1

# Example: binary classification

$$H(Y) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

- Where $p_i = p(Y = y_i)$, and $m$ is the number of classes
- **Example:** The entropy of Y:
  - 50% of **Yes** and 50% of **No**
  - $H(Y) = -(0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)) = 1$

P(Y = Yes) P(Y = No)

| X | Y |
|---|---|
| Math | Yes |
| History | No |
| CS | Yes |
| Math | No |
| Math | No |
| CS | Yes |
| History | No |
| Math | Yes |

# Conditional Entropy: H(Y | X)

- H(Y | X): the average conditional entropy of Y.

$$H(Y|X) = \sum_i p(X = v_i) * H(Y|X = v_i)$$

- From data, we calculate $p(X = v_i)$:
  - $p(X = \text{Math}) = {}^4/_8 = 0.5$
  - $p(X = \text{History}) = {}^2/_8 = 0.25$
  - $p(X = \text{CS}) = {}^2/_8 = 0.25$

| $v_i$ | $p(X = v_i)$ | $H(Y\|X = v_i)$ |
|---|---|---|
| Math | 0.5 | ? |
| History | 0.25 | ? |
| CS | 0.25 | ? |

| X | Y |
|---|---|
| Math | Yes |
| History | No |
| CS | Yes |
| Math | No |
| Math | No |
| CS | Yes |
| History | No |
| Math | Yes |

For history and CS, the Y value is certain, For math, the yes and no are equally likely

# Specific Conditional Entropy: H(Y | X=$v_i$)

- $H(Y|X = v_i)$: entropy of Y among only those records in which X has value $v_i$.

| X | Y |
|---|---|
| Math | Yes |
| Math | No |
| Math | No |
| Math | Yes |

| X | Y |
|---|---|
| History | No |
| History | No |

| X | Y |
|---|---|
| CS | Yes |
| CS | Yes |

- From the data, we obtain:
  - $H(Y|X = \text{Math}) = -(^2/_4 * \log_2(^2/_4) + ^2/_4 * \log_2(^2/_4)) = 1$
  - $H(Y|X = \text{History}) = -(^0/_2 * \log_2(^0/_2) + * ^2/_2 \log_2(^2/_2)) = 0$
  - $H(Y|X = \text{CS}) = -(^2/_2 * \log_2(^2/_2) + ^0/_2 * \log_2(^0/_2)) = 0$

For history and CS, the Y value is certain, For math, the yes and no are equally likely

# Conditional Entropy: H(Y | X)

| $v_i$ | $p(X = v_i)$ | $H(Y\|X = v_i)$ |
|---|---|---|
| Math | 0.5 | 1 |
| History | 0.25 | 0 |
| CS | 0.25 | 0 |

$$H(Y|X) = \sum_i p(X = v_i) * H(Y|X = v_i)$$
$$= 0.5 * 1 + 0.25 * 0 + 0.25 * 0$$
$$= 0.5$$

# Information Gain (IG)

$$IG(Y|X) = H(Y) - H(Y|X)$$

- From the example:
  - $H(Y) = 1$
  - $H(Y|X) = 0.5$
- Thus:
  - $IG(Y|X) = 1 - 0.5 = 0.5$

# Test your understanding

- A decision tree node contains 20 training examples: 10 positive and 10 negative. What can we say about its entropy?
  - A. It is 0 because the node has many examples
  - B. It is high because the classes are evenly mixed
  - C. It is low because there are only two classes
  - D. It depends only on the number of features

$$-(0.5 * log_2 0.5 + 0.5 * log_2 0.5) = -\left(log_2 \frac{1}{2}\right) = -(-) = 1$$

# Test your understanding

- A node contains 15 examples, all belonging to the same class. What is its entropy?
  - A. 0
  - B. 0.5
  - C. 1
  - D. Cannot be determined

# Test your understanding

- For a binary class label, when is entropy highest?
  - A. when all examples are positive
  - B. when all examples are negative
  - C. when positive and negative examples are equally balanced
  - D. when there are more positive than negative

# Test your understanding

- A node has 100 examples: 50 positive and 50 negative. Which split is likely better?
  - A. Left: 25 positive, 25 negative; Right: 25 positive, 25 negative
  - B. Left: 50 positive, 0 negative; Right: 0 positive, 50 negative
  - C. Left: 45 positive, 45 negative; Right: 5 positive, 5 negative
  - D. Left: 30 positive, 30 negative; Right: 20 positive, 20 negative

# Test your understanding

- A split produces:
  - Left child: 10 positive, 0 negative
  - Right child: 40 positive, 50 negative
- Which statement is most accurate?
  - A. The split is useless because it is unbalance
  - B. The split reduces some uncertainty because one child is pure
  - C. The split must be the best split because one child is pure
  - D. The split has zero information gain

# Test your understanding

- For a numeric feature in a decision tree, should we always split at the median value?
  - A. Yes, because it creates equal-sized groups
  - B. Yes, because equal-sized groups always maximise information gain
  - C. No, because the best threshold is the one that most reduces class uncertainty
  - D. No, because decision trees cannot use numeric features

# Test your understanding

- A node has 100 examples: 50 positive and 50 negative.
  - Split A:
    - Left: 50 examples, 25 positive / 25 negative
    - Right: 50 examples, 25 positive / 25 negative
  - Split B:
    - Left: 20 examples, 20 positive / 0 negative
    - Right: 80 examples, 30 positive / 50 negative
- Which is better according to information gain?
  - A. Split A, because it is balanced
  - B. Split B, because it reduces uncertainty more
  - C. They are equally good
  - D. Cannot say because the split must be 50/50

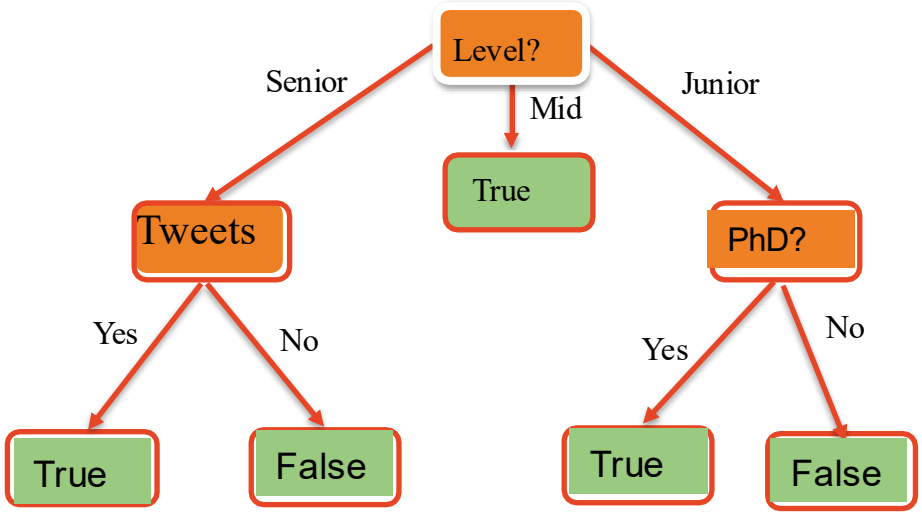
The information gain of split A is 0

For split B, the entropy of left child is 0

For split B, the entropy of right child is less than 1

For split B, there is some information gain

# Is the previous decision tree good?




- Let's check whether the split on **Level** attribute is good.
- We need to show that Level attribute has the **highest information gain**.

# Calculation

- $H(\text{Interviewed well}) = H(9,5) = -({}^{9}/_{14}\log_2({}^{9}/_{14}) + {}^{5}/_{14}\log_2({}^{5}/_{14})) = 0.94$
- $H(\text{Interviewed well} \mid \text{Level}) = \sum_i p(\text{Level} = v_i) * H(\text{Interviewed well} \mid \text{Level} = v_i)$

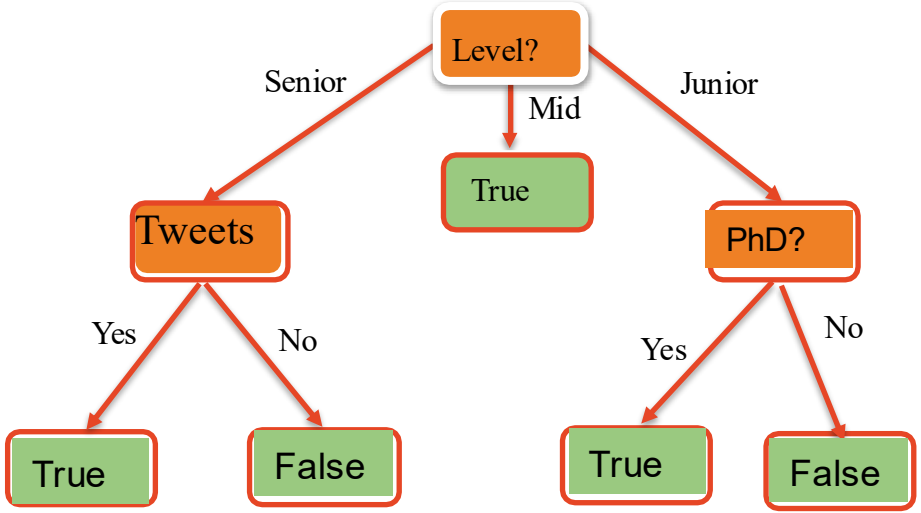
| $v_i$ | $p(\text{Level} = v_i)$ | $H(\text{Interviewed well} \mid \text{Level} = v_i)$ |
|---|---|---|
| Senior | ${}^{5}/_{14} = 0.36$ | $H(2,3) = -\left({}^{2}/_{5} * \log_2\left({}^{2}/_{5}\right) + {}^{3}/_{5} * \log_2\left({}^{3}/_{5}\right)\right) = 0.97$ |
| Mid | ${}^{4}/_{14} = 0.29$ | $H(4,0) = -({}^{4}/_{4} * \log_2({}^{4}/_{4}) + {}^{0}/_{4} * \log_2({}^{0}/_{4})) = 0$ |
| Junior | ${}^{5}/_{14} = 0.36$ | $H(3,2) = -\left({}^{3}/_{5} * \log_2\left({}^{3}/_{5}\right) + {}^{2}/_{5} * \log_2\left({}^{2}/_{5}\right)\right) = 0.97$ |

- Then:
  - $H(\text{Interviewed well} \mid \text{Level}) = 0.36 * 0.97 + 0.29 * 0 + 0.3 * 0.97 = 0.7$
- Thus:
  - $IG(\text{Interviewed well} \mid \text{Level}) = H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{Level})$
    $= 0.94 - 0.7 = 0.24$

# Calculation

- Similarly:
  - $IG(\text{Interviewed well} \mid \text{Tweets})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{Tweets}) = 0.15$
  - $IG(\text{Interviewed well} \mid \text{PhD})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{PhD}) = 0.048$
  - $IG(\text{Interviewed well} \mid \text{Lang})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{Lang}) = 0.029$
- Level has the highest information gain, therefore it was good to choose that attribute.

# Is the previous decision tree good?




- Let's also check whether the split on **PhD** attribute is good.
- We need to show that PhD attribute has the **highest information gain**.

# PhD attribute – subset of 5 records with Junior level

| | Level | Lang | Tweets | PhD | Interviewed well |
|---|---|---|---|---|---|
| 4 | Junior | Python | No | No | True |
| 5 | Junior | R | Yes | No | True |
| 6 | Junior | R | Yes | Yes | False |
| 10 | Junior | Python | Yes | No | True |
| 14 | Junior | Python | No | Yes | False |

- $H(\text{Interviewed well}) = H(3,2) = -({}^{3}/_{5}\log_2({}^{3}/_{5}) + {}^{2}/_{5}\log_2({}^{2}/_{5})) = 0.97$
- $H(\text{Interviewed well} \mid \text{PhD}) = \sum_i p(\text{PhD} = v_i) * H(\text{Interviewed well} \mid \text{PhD} = v_i)$

| $v_i$ | $p(\text{PhD} = v_i)$ | $H(\text{Interviewed well} \mid \text{PhD} = v_i)$ |
|---|---|---|
| Yes | ${}^{2}/_{5} = 0.4$ | $H(0,2) = 0$ |
| No | ${}^{3}/_{5} = 0.6$ | $H(3,0) = 0$ |

- Then: $H(\text{Interviewed well} \mid \text{PhD}) = 0$

# Calculation

- Then, the Information gain for each attribute:
  - $IG(\text{Interviewed well} \mid \text{PhD})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{PhD}) = 0.97$
  - $IG(\text{Interviewed well} \mid \text{Tweets})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{Tweets}) = 0.02$
  - $IG(\text{Interviewed well} \mid \text{Lang})$
    $= H(\text{Interviewed well}) - H(\text{Interviewed well} \mid \text{Lang}) = 0.02$
- PhD has the highest information gain, therefore it was good to choose that attribute next for the Junior Level.

# Train a decision tree classifier in scikit-learn

```
from sklearn.tree import DecisionTreeClassifier

# Let's fit a model
tree = DecisionTreeClassifier(max_depth=2, criterion='entropy')
tree.fit(X_train, Y_train)
```

## Some important parameters:

- **max_depth:** the maximum depth of the tree.
- **criterion:**
  - **gini**: choose splits that minimise misclassification.
  - **entropy**: choose splits that minimise total uncertainty.
- **splitter:**
  - **best**: choose the optimal threshold for each feature.
  - **random**: choose the best random threshold for each feature.

# Test your understanding

- A dataset has four equally common classes: A, B, C and D. At the root node, a split produces:
  - Left branch: examples from classes A and B only
  - Right branch: examples from classes C and D only
- Which statement is most accurate?
  - A. This is useless because each branch still contains more than one class
  - B. This gives some information gain because each branch is less uncertain than the original node
  - C. This gives maximum information gain because the classes are evenly divided
  - D. This must be worse than a 50/50 random split

# Let's calculate the IG

- Entropy before the split
  - $log_2 4 = 2$
- Entropy after the split:
  - 0.5 *1 + 0.5*1 = 1
  - IG = 2-1 = 1

| Branch | Probability of landing in this branch | Conditional Entropy |
|---|---|---|
| Left | 0.5 | 1 |
| right | 0.5 | 1 |

# Evaluating a Decision Tree



# Why do we need to evaluate a decision tree

- Like many other machine learning algorithms, decision tree algorithms like ID3 is greedy
  - Choose the best split locally ( at the current step)
  - Does not guarantee a globally optimal tree
- Decision tress can overfit
  - Keep splitting until training data is perfectly classified
  - may capture noise instead of patterns
- Model depends on training data
  - Different samples leads to different trees
  - Results may not generalize to new data
- Real goal:
  - perform well on **unseen data**, not just training data

# Training Accuracy vs Test Accuracy

- Training performance is not enough
  - High training accuracy and low test accuracy can indicate overfitting
- Controlling complexity of a tree
  - `max_depth`
  - **min_samples_leaf:** The minimum number of samples that each leaf node must contain after a split.
  - **min_samples_split:** The minimum number of samples required at a node before the tree is allowed to split it.
- Those parameters can be tuned by trying various parameter combinations with cross-validation to choose the best combination

| Model | Training accuracy | Test accuracy |
|---|---|---|
| Very deep tree | 1.00 | 0.70 |
| Smaller tree | 0.85 | 0.80 |

# Confusion matrix in binary classifier

- Confusion matrix shows what kinds of mistakes a classifier make

Interviewed well:

| | Predicted True | Predicted False |
|---|---|---|
| Actual True | True Positive (TP) | False Negative (FN) |
| Actual False | False Positive (FP) | True Negative (TN) |

$$\text{Accuracy}=\frac{TP+TN}{TP+FN+FP+TN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{F1} = \frac{2*Precision*Recall}{Precision+Recall}$$

among predicted good applicants, how many were actually good?

among all good applicants, how many did the model find

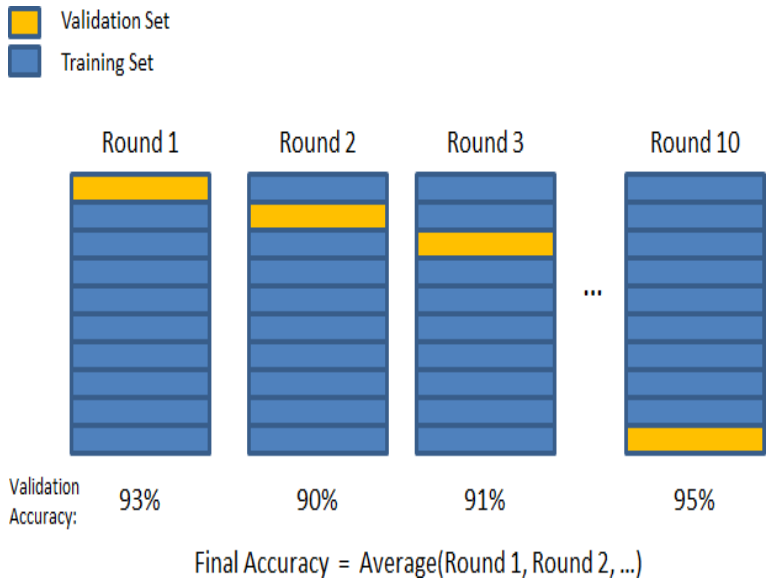
# Confusion matrix for multi-class classifier

– For multi-class classification, we can treat each class as "positive" in turn, or use a larger confusion matrix with one row/column per class.

| | Predicted setosa | Predicted versicolor | Predicted virginica |
|---|---|---|---|
| Actual setosa | 16 | 0 | 0 |
| Actual versicolor | 0 | 16 | 1 |
| Actual virginica | 0 | 0 | 17 |

# Cross - Validation

- A single train/test split may be lucky or unlucky
- In $k$-fold cross-validation:
  - split the training data into $k$ folds
  - train $k$ temporary models
  - average the results



# Using Cross-Validation to Select Parameters

- We can use cross-validation to select important model parameters (tune a tree when the model is a decision tree)
- If we want to choose the maximum tree depth and minimal sample leaf parameter
- We can use **`GridSearchCV`** from sklearn to find the best value

```
penguin_tree_param_grid = {
    "model__max_depth": [2, 3, 4, 5, None],
    "model__min_samples_leaf": [1, 3, 5, 10, 20],
}

penguin_tree_search = GridSearchCV(
    penguin_tree,
    param_grid=penguin_tree_param_grid,
    cv=cv,
    scoring="balanced_accuracy",
    n_jobs=1
)

penguin_tree_search.fit(X_penguin_train, y_penguin_train)
```

```
penguin_tree = make_penguin_model(DecisionTreeClassifier(
    criterion="entropy",
    random_state=RANDOM_STATE
))
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
```

# The final evaluation workflow

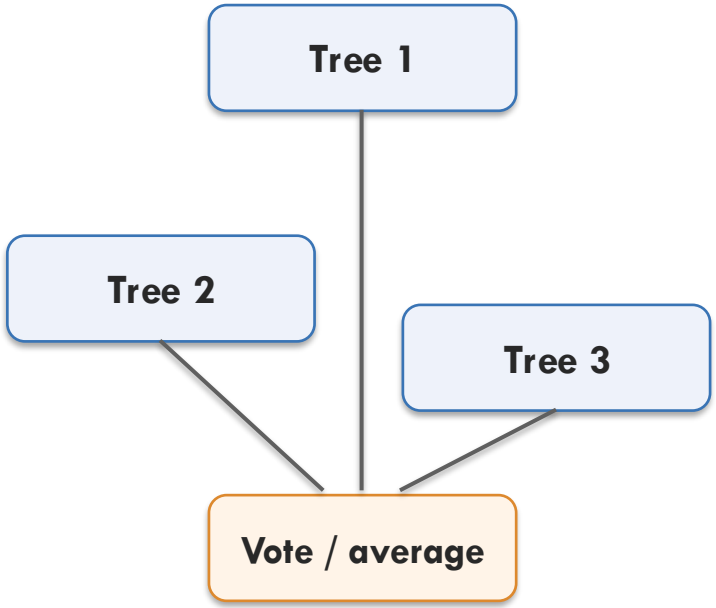
- Do a train/test split
- For the training data
  - Try several tree settings
  - Use cross-validation to compare them
  - Choose the best setting
  - Train final tree on all training data
- Evaluate once on held-out test data

# Beyond Single Decision Trees



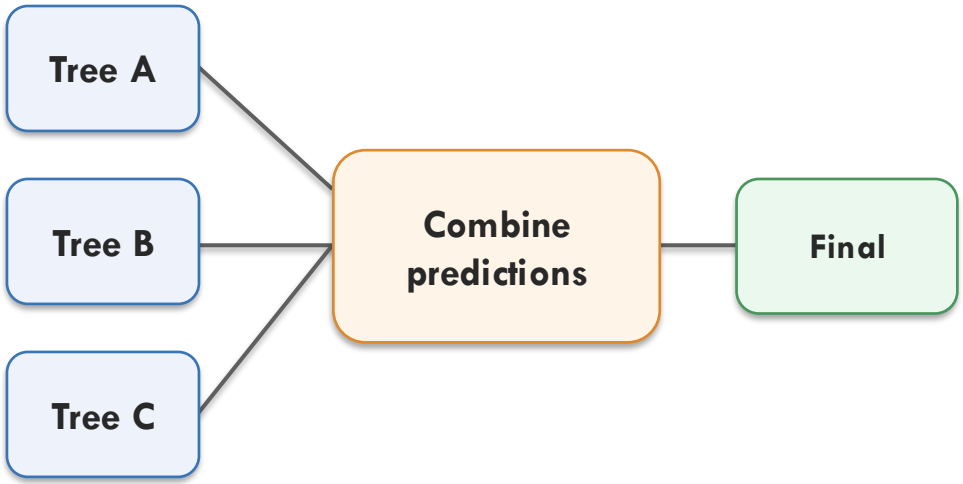
# Why consider ensembles of trees

- A single decision tree is easy to interpret, but can be unstable.
- Small changes in the training data may produce a different tree.
- Deep trees often fit the training data very well, but generalise poorly.
- Ensembles combine many trees to improve predictive performance.



# Ensemble learning: the basic idea

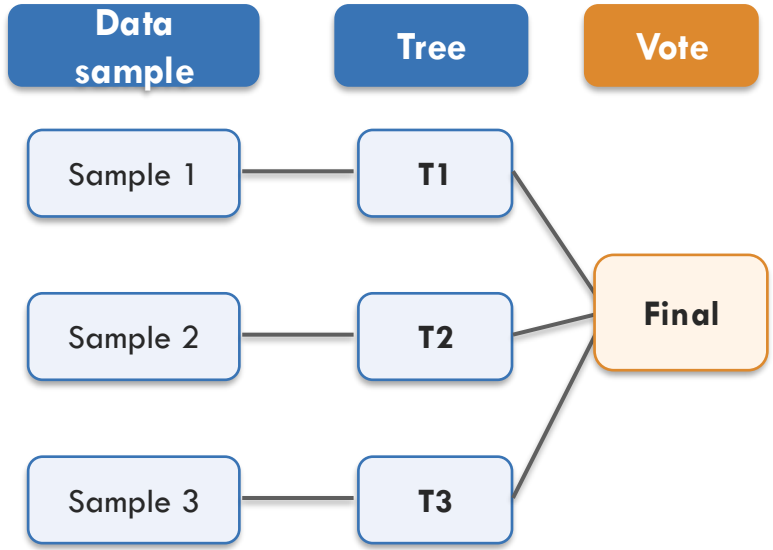
- Train multiple models instead of one.
- Combine their predictions.
- The group can be more reliable than any single model.
- For trees, the individual models are usually decision trees.




Classification: majority vote
Regression: average

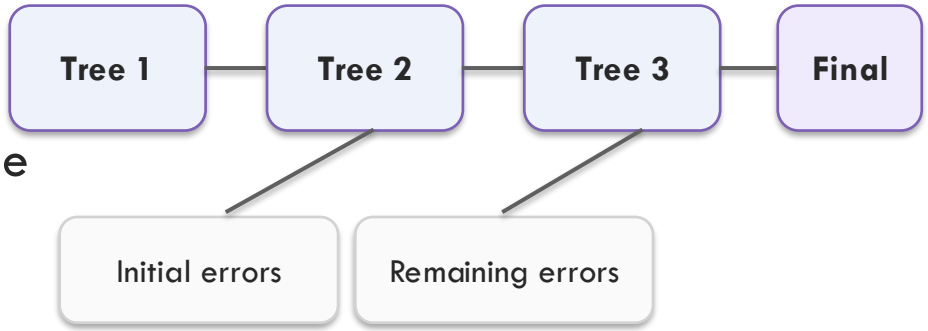
# Random Forest: many diverse trees

- Start with many bootstrap samples of the training data.
- Train one tree on each sample.
- At each split, consider only a random subset of features.
- Combine predictions by voting or averaging.
- Main effect: reduce variance compared with one deep tree.



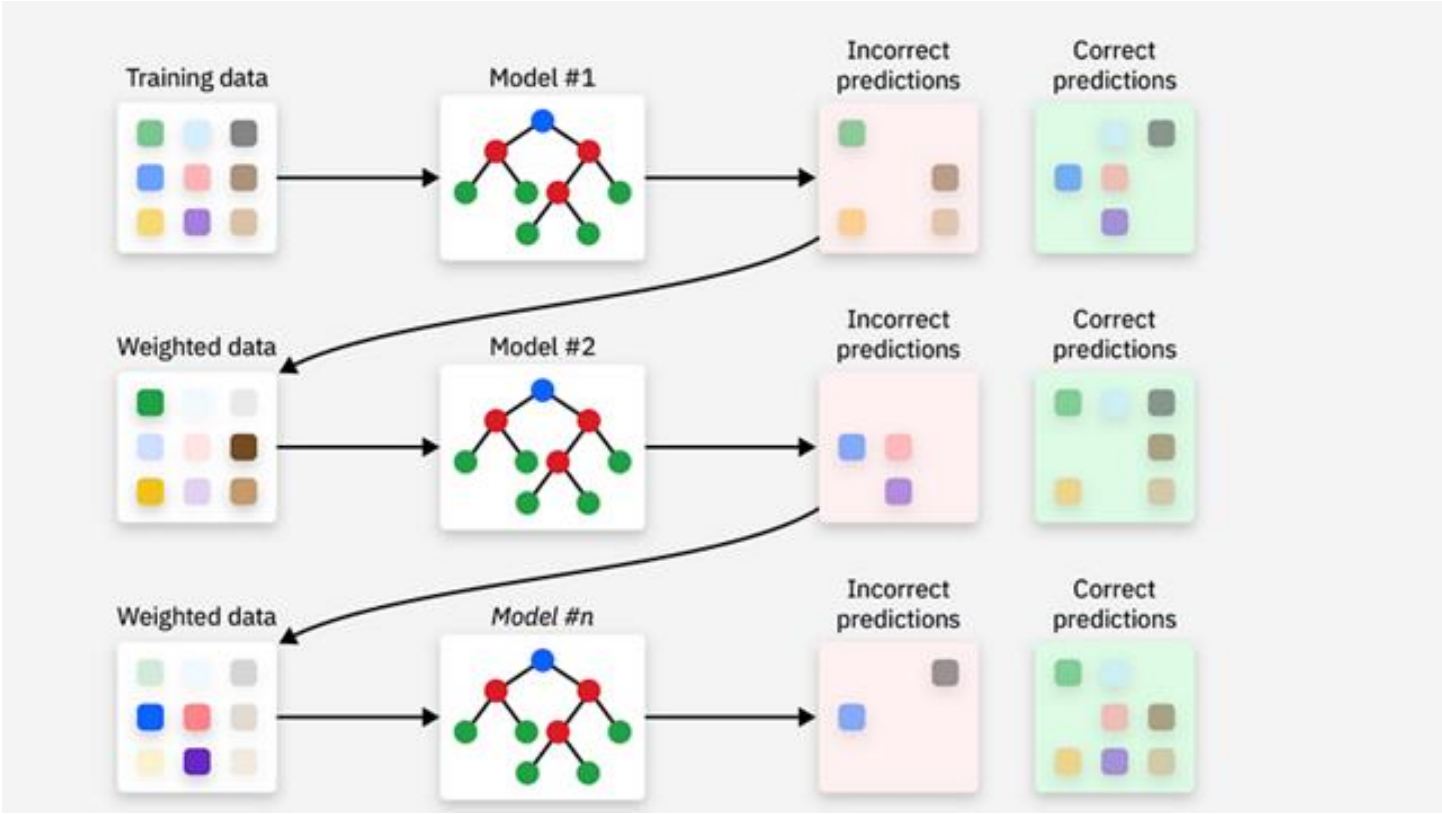
# Gradient Boosting: trees that correct early erros

- Build trees sequentially, not independently.
- Each new tree focuses on what the current ensemble still gets wrong.
- Predictions are added together step by step.
- Often very accurate, but more sensitive to tuning than random forest.




Key intuition: each tree is a small correction to the current model.

# AdamBoost example



# Comparing tree models

| | Single Tree | Random Forest | Gradient Boosting |
|---|---|---|---|
| Training | One tree | Many independent trees | Trees built sequentially |
| Main strength | Interpretability | Stable, strong baseline | Often high accuracy |
| Main risk | Overfitting | Less interpretable | Sensitive to tuning |
| Useful params | max_depth<br>min_samples_leaf | n_estimators<br>max_features | learning_rate<br>n_estimators |

# REVIEW

# W10 review: Decision tree

- Objective
- Learn techniques for supervised machine learning, with tools in Python.

- Lecture
- Decision tree
- Information gain
- Evaluation setup
- Tree ensembles: random forest and gradient boosting

**Readings**

- Data Science from Scratch, Ch. 17

**Exercises**

- sklearn: decision tree
- sklearn: random forest
- sklearn: gradient boosting

# On good data science

- How to evaluate machine learning models: http://blog.dato.com/how-to-evaluate-machine-learning-models- the-pitfalls-of-ab-testing
- Top 10 data science practitioner pitfalls: http://www.slideshare.net/0xdata/top-10-data-science-practitioner-pitfalls
- Introduction to Applied Machine Learning: Generalisation: http://www.inf.ed.ac.uk/teaching/courses/iaml/slides/eval-2x2.pdf